

Guida Discorsiva di Studio

Sistemi Intelligenti

Con esempi, analogie e spiegazioni intuitive

A.A. 2025–2026

Indice

1	Introduzione all’Intelligenza Artificiale	3
1.1	I quattro modi di vedere l’IA	3
1.2	Agenti intelligenti: come funzionano	4
2	Ricerca Non Informata (Blind Search)	5
2.1	Il problema della ricerca	5
2.2	BFS – Ricerca in Ampiezza	5
2.3	DFS – Ricerca in Profondità	6
2.4	IDDFS – Approfondimento Iterativo	6
2.5	UCS – Ricerca a Costo Uniforme	7
3	Ricerca Informata (A* e Euristiche)	8
3.1	L’idea dell’euristica	8
3.2	Algoritmo A*	8
4	Ricerca Avversariale (Giochi)	9
4.1	Minimax	9
4.2	Potatura Alpha-Beta	10
4.3	Tecniche avanzate	11
5	Problemi di Soddisfacimento di Vincoli (CSP)	11
5.1	Cos’è un CSP	11
5.2	Propagazione dei vincoli	12
5.3	Backtracking con euristiche	12
6	Logica Proposizionale	13

6.1	Perché la logica	13
6.2	Dimostrazione di teoremi	13
6.3	Clausole di Horn e Forward/Backward Chaining	14
7	Logica del Primo Ordine (FOL)	14
7.1	Perché la FOL	14
7.2	Unificazione	15
7.3	Modus Ponens Generalizzato e Skolemizzazione	16
8	Ontologie e Rappresentazione della Conoscenza	16
9	Machine Learning	17
9.1	Alberi di Decisione	17
10	Reti Neurali	18
10.1	Il Percettrone	18
10.2	MLP e Backpropagation	19
11	Apprendimento per Rinforzo	19
11.1	Il framework MDP	19
11.2	Algoritmi di apprendimento	20
12	Riepilogo – Mappa concettuale	21

1 Introduzione all'Intelligenza Artificiale

1.1 I quattro modi di vedere l'IA

Cosa vuol dire “intelligenza artificiale”?

Immagina di dover costruire una macchina “intelligente”: in base a come definisci l'intelligenza, costruirai cose molto diverse. Il campo dell'IA si divide proprio su questo punto, e possiamo identificare quattro grandi visioni:

- 1. Agire umanamente** – La macchina è intelligente se si comporta come un essere umano, al punto che non riesci a distinguerla da uno. Questo è il **Test di Turing (1950)**: conversi via tastiera con due interlocutori, uno umano e uno macchina, e non sai chi è chi. Se la macchina ti inganna, ha “superato” il test.
- 2. Pensare umanamente** – Non basta comportarsi come un umano; bisogna anche *ragionare* come un umano, avere gli stessi processi mentali (cognitivismo, psicologia cognitiva). Meno utile per l'ingegneria.
- 3. Pensare razionalmente** – Usare la logica formale per dedurre conclusioni corrette. Il problema è che il mondo reale è spesso incerto e non tutto si formalizza in logica.
- 4. Agire razionalmente** – Il libro di testo (e il corso) adotta questa visione: un **agente razionale** è un sistema che, date le informazioni a disposizione, compie le azioni che *massimizzano la misura di prestazione attesa*. Non importa come ragiona internamente: importa che faccia la cosa giusta.

Analogia: Il pilota automatico

Un agente razionale è come il pilota automatico di un aereo: non “pensa” come un pilota umano, non ha paura, non ha intuizioni. Ma dato l'obiettivo (arrivare a destinazione in sicurezza) e le informazioni (quota, vento, carburante), sceglie le azioni che massimizzano il successo. Non dobbiamo chiederci “ragiona come un umano?” ma “raggiunge gli obiettivi nel modo migliore possibile?”

Da ricordare: La distinzione IA Forte vs Debole

- **IA Debole**: la macchina *simula* l'intelligenza ma non ha vera comprensione. È quello che abbiamo oggi (ChatGPT, AlphaGo...).
- **IA Forte**: la macchina ha vera comprensione, coscienza, stati mentali come gli umani. È ancora fantascienza.
- **Argomento della Stanza Cinese (Searle)**: immagina di essere chiuso in una stanza, ti passano foglietti con simboli cinesi, tu segui un manuale di regole e restituisci simboli. Chi ti vede dall'esterno pensa che tu capisca il cinese, ma tu non hai idea di cosa significhino. Questa è l'IA debole: manipola simboli senza capirli.

1.2 Agenti intelligenti: come funzionano

Anatomia di un agente

Qualsiasi sistema IA è un **agente**: percepisce l'ambiente tramite **sensori**, elabora le informazioni, e agisce tramite **attuatori**. Un robot aspirapolvere percepisce lo sporco con sensori ottici e aspira con il motore. Un motore di scacchi percepisce la scacchiera (input) e muove i pezzi (output).

Il **programma agente** è la funzione che mappa la storia delle percezioni (la **sequenza percettiva**) a un'azione. L'agente è **razionale** se questa funzione massimizza la misura di prestazione.

Importante: razionalità $\neq$ onniscienza. L'agente fa del meglio con quello che sa; non può sapere tutto.

I cinque tipi di agenti (dal più semplice al più potente)

1. **Agente reattivo semplice**: risponde direttamente alle percezioni con regole “se-allora”. Il termostato: *se* temperatura < 20 *allora* accendi il riscaldamento. Velocissimo, ma non ha memoria – ogni percezione è trattata come se fosse la prima.
2. **Agente reattivo con modello**: mantiene uno **stato interno** del mondo. È come avere una mappa mentale: anche se non vedo direttamente cosa c'è dietro la curva, so che lì c'era un'auto ferma (l'ho vista 5 secondi fa). Funziona in ambienti parzialmente osservabili.
3. **Agente goal-driven** (basato su obiettivi): non solo sa come è il mondo, ma sa *dove vuole arrivare*. Pianifica sequenze di azioni per raggiungere il goal. Il navigatore GPS: conosce la mappa e la destinazione.
4. **Agente utility-driven** (basato sull'utilità): ha una funzione di utilità che quantifica quanto è “buono” ogni stato. Non solo “arrivo a destinazione” ma “arrivo nel minor tempo, con meno carburante, evitando il traffico”. Gestisce trade-off e incertezza.
5. **Agente capace di apprendere**: parte da conoscenza iniziale limitata e migliora con l'esperienza. Composto da: *critico* (“stai andando bene?”), *modulo di apprendimento* (“aggiusta il comportamento”), *generatore di problemi* (“esplora cose nuove”).

Esempio: Classificare gli ambienti

Prima di scegliere il tipo di agente giusto, bisogna capire l'ambiente:

- **Completamente osservabile**: l'agente vede tutto lo stato. Scacchi: sì (vedi tutta la scacchiera). Poker: no (non vedi le carte degli altri).
- **Deterministico**: le azioni hanno effetti certi e prevedibili. Un braccio robotico in fabbrica: sì. Guida su strada bagnata: no (stocastico).
- **Episodico**: ogni episodio è indipendente dagli altri. Classificare immagini: sì (classificare un gatto non dipende dal gatto precedente). Scacchi: no (le mosse dipendono dalla storia della partita).
- **Statico**: il mondo non cambia mentre l'agente ragiona. Cruciverba: sì. Guida autonoma: no (dinamico).

In ambienti **deterministici** bastano sistemi ad **anello aperto** (esegui il piano senza rileggere). In ambienti **stocastici** servono sistemi ad **anello chiuso** (monitora e riadatta continuamente).

2 Ricerca Non Informata (Blind Search)

2.1 Il problema della ricerca

Come si formula un problema di ricerca

Molti problemi si riducono a “trovare il percorso giusto in uno spazio di stati”. Formalmente un problema ha:

- **Stato iniziale:** da dove partiamo.
- **Azioni:** cosa possiamo fare in ogni stato.
- **Modello di transizione:** dove arriviamo applicando un'azione.
- **Test obiettivo:** come sappiamo di aver finito.
- **Costo del cammino:** quanto costa ogni sequenza di azioni.

La soluzione è la sequenza di azioni che porta dallo stato iniziale all'obiettivo.

Gli algoritmi di ricerca costruiscono un **albero di ricerca** sullo spazio degli stati: la radice è lo stato iniziale, i rami sono le azioni, i nodi figli sono gli stati successivi. La **frontiera** (o fringe) è l'insieme dei nodi generati ma non ancora “esplorati” (espansi). La differenza tra gli algoritmi sta nel *quale nodo della frontiera espandere per primo*.

Da ricordare: Le quattro metriche degli algoritmi

- **Completezza:** trova sempre una soluzione se esiste?
- **Ottimalità:** trova la soluzione di costo minimo?
- **Complessità temporale:** quanti nodi genera/espande?
- **Complessità spaziale:** quanta memoria usa (nodi in memoria contemporanea-mente)?

Parametri da ricordare: b = fattore di ramificazione (figli per nodo), d = profondità della soluzione ottima, m = profondità massima dell'albero.

2.2 BFS – Ricerca in Ampiezza

Come funziona BFS

BFS espande i nodi **livello per livello**: prima la radice, poi tutti i suoi figli (profondità 1), poi tutti i loro figli (profondità 2), e così via. Usa una coda **FIFO** (First In First Out): il primo nodo inserito è il primo ad essere espanso.

Analogia: Cerchi sull'acqua

Butta un sasso in uno stagno: le onde si espandono in cerchi concentrici, colpendo prima le rive vicine e poi quelle lontane. BFS fa esattamente questo: esplora prima gli stati “vicini” (pochi passi dall'inizio) e poi quelli “lontani”.

Esempio: BFS su un grafo semplice

Grafo: $A - B - D$, $A - C - E$, $D - F$ (il goal è F).

1. Frontiera: [A]. Espando $A \rightarrow$ genero B, C.

2. Frontiera: [B, C]. Espando B $\rightarrow$ genero D.
3. Frontiera: [C, D]. Espando C $\rightarrow$ genero E.
4. Frontiera: [D, E]. Espando D $\rightarrow$ genero F. **GOAL trovato!**

BFS garantisce di trovare il cammino con meno passi ($d=3$: $A \rightarrow B \rightarrow D \rightarrow F$). Però ha usato memoria per tenere tutti i nodi a ogni livello.

Da ricordare: BFS in breve

Completo (sì, se b finito). Ottimale (sì, se costi uniformi). Tempo: $O(b^{d+1})$. Spazio: $O(b^{d+1})$ – il collo di bottiglia. Con $b = 10$ e $d = 6$: oltre 10 milioni di nodi in memoria!

2.3 DFS – Ricerca in Profondità

Come funziona DFS

DFS espande sempre il nodo **più in profondità** nella frontiera. Usa uno stack LIFO (Last In First Out): l'ultimo inserito è il primo espanso. Scende fino in fondo lungo un ramo, poi fa backtrack se non trova il goal.

Analogia: Esplorare una grotta

Entri in una grotta con molti corridoi. DFS prende sempre il primo corridoio a sinistra, continua fino al fondo, poi torna indietro e prova il corridoio successivo. BFS invece esplorerebbe tutti i corridoi al primo incrocio prima di andare avanti.

Esempio: Perché DFS usa poca memoria

In un albero con $b = 3$ e $d = 4$: DFS tiene in memoria solo il cammino corrente + i fratelli non espansi. Al massimo $b \times m$ nodi contemporaneamente. BFS invece tiene tutti i nodi del livello corrente: b^d nodi. Per $b = 10, m = 10$: DFS usa 100 nodi, BFS usa 10^{10} nodi!

Attenzione: Quando DFS fallisce

DFS **non è completo** (può ciclare all'infinito in grafi con cicli, se non controlli i nodi visitati) e **non è ottimale** (trova la prima soluzione, non la migliore). In un grafo con cicli, DFS senza controllo dei cicli va in loop.

2.4 IDDFS – Approfondimento Iterativo

Il meglio dei due mondi

IDDFS (Iterative Deepening DFS) combina il **poco uso di memoria di DFS** con la **completezza e ottimalità di BFS**. Come? Esegue ripetutamente DFS con un limite di profondità crescente: prima cerca solo fino a profondità 0, poi fino a 1, poi 2, ecc. Sembra uno spreco ripetere – ma i nodi ripetuti sono i livelli superficiali, che sono pochi rispetto all'ultimo livello. Per $b = 10$: il livello d ha 10^d nodi, i livelli da 0 a $d - 1$ ne hanno circa $\frac{10^d}{9}$ (trascurabile).

Esempio: IDDFS con limite $l = 0,1,2$

Goal a profondità 2 in un albero con $b = 2$:

$l=0$: espando solo la radice A. Non trovato.

$l=1$: espando $A \rightarrow B$, $A \rightarrow C$. Non trovato.

$l=2$: espando $A \rightarrow B \rightarrow D$, $A \rightarrow B \rightarrow E$, $A \rightarrow C \rightarrow F$ (GOAL!).

Sì, D ed E sono stati esplorati “invano” al passo precedente, ma la ridondanza è minima.

Da ricordare: IDDFS è il re della blind search

Completo: sì. Ottimale: sì (con costi uniformi). Tempo: $O(b^d)$. Spazio: $O(b \cdot d)$ – lineare come DFS. È l'**algoritmo blind preferito** in pratica.

2.5 UCS – Ricerca a Costo Uniforme

Quando i costi sono diversi

BFS trova il cammino con meno passi, ma se ogni passo ha un costo diverso? UCS (Uniform Cost Search) espande sempre il nodo con il **costo cumulativo** $g(n)$ **più basso**, usando una coda con priorità. È praticamente l'algoritmo di Dijkstra applicato alla ricerca con goal.

Esempio: UCS vs BFS con costi diversi

Grafo: $A \xrightarrow{1} B \xrightarrow{1} C$ (goal), $A \xrightarrow{10} C$ (goal).

BFS trova $A \rightarrow C$ (2 mosse) che costa 10.

UCS trova $A \rightarrow B \rightarrow C$ (3 mosse) che costa 2.

BFS ottimizza il numero di passi, UCS ottimizza il costo.

Algoritmo	Completo	Ottimale	Tempo	Spazio
BFS	Sì	Sì*	$O(b^{d+1})$	$O(b^{d+1})$
DFS	No	No	$O(b^m)$	$O(bm)$
IDDFS	Sì	Sì*	$O(b^d)$	$O(bd)$
UCS	Sì	Sì	$O(b^{1+C^*/\epsilon})$	uguale

*Con costi uniformi.

3 Ricerca Informata (A* e Euristiche)

3.1 L'idea dell'euristica

Usare la conoscenza del dominio

La blind search è “cieca”: non sa in quale direzione si trova il goal. La ricerca informata usa una funzione **euristica** $h(n)$ che *stima* quanto siamo lontani dal goal. Con questa guida, possiamo concentrare la ricerca nelle direzioni più promettenti.

Un'euristica è **ammissibile** se non *sovrastima mai* il costo reale: $h(n) \leq h^*(n)$. Questo garantisce che A* trovi la soluzione ottimale. Un'euristica è **consistente (monotona)** se soddisfa la disuguaglianza triangolare: $h(n) \leq c(n, n') + h(n')$.

Esempio: Euristiche per il problema dell'8-puzzle

Il problema: una griglia 3×3 con 8 tasselli numerati e una casella vuota. Devi raggiungere la configurazione goal muovendo i tasselli.

Due euristiche classiche:

- h_1 = numero di tasselli **fuori posto**. Se 4 tasselli non sono nella posizione corretta, $h_1 = 4$. È ammissibile perché ogni tassello richiede almeno 1 mossa.
- h_2 = **distanza di Manhattan**: somma delle distanze (orizzontale + verticale) di ogni tassello dalla posizione corretta. È ammissibile perché i tasselli non si possono teleportare.

$h_2 \geq h_1$ sempre $\Rightarrow h_2$ **domina** $h_1 \Rightarrow$ A* con h_2 espande meno nodi (la euristica più informata, meglio è).

3.2 Algoritmo A*

Come funziona A*

A* ordina la frontiera per $f(n) = g(n) + h(n)$:

- $g(n)$: costo effettivo per arrivare da start a n (quanto ho già speso).
- $h(n)$: stima euristica da n al goal (quanto prevedo di spendere ancora).
- $f(n)$: stima del costo totale del miglior cammino che passa per n .

A* è una **fusione intelligente**: UCS usa solo g (conosce il passato ma non il futuro), Greedy usa solo h (guarda il futuro ma ignora il passato). A* usa entrambi.

La **frontiera di A*** forma ellissi orientate verso il goal (non cerchi concentrici come UCS): esplora “di più” nelle direzioni dove f è basso, cioè dove il percorso totale stimato è migliore.

Esempio: A* su una mappa numerica

Mappa con 5 città. Start = A, Goal = E.

Nodo	$g(n)$ (costo reale da A)	$h(n)$ (stima al goal)
A	0	10
B	3	8
C	5	6
D	4	9
E	12	0

Passo 1: espando A ($f = 0 + 10 = 10$). Genero B ($f = 3 + 8 = 11$) e D ($f = 4 + 9 = 13$).

Passo 2: espando B ($f = 11$, il minore). Genero C ($g = 3 + 2 = 5$, $f = 5 + 6 = 11$).

Passo 3: B e C hanno entrambi $f = 11$; espando C. Genero E ($g = 5 + 7 = 12$, $f = 12 + 0 = 12$).

Passo 4: espando D ($f = 13$)... ma $f(D) > f(E)$, quindi prima espandiamo E. **GOAL!**
Costo = 12.

A* ha ignorato D perché sapeva già che nessun percorso passante per D poteva essere migliore.

Attenzione: Ammissibilità vs Consistenza

- Su **alberi** (senza ricicli): basta l'ammissibilità per garantire l'ottimalità di A*.
- Su **grafi** (con lista chiusa dei visitati): serve la consistenza. Consistente $\Rightarrow$ ammissibile, ma non viceversa.
- Se usi A* su grafi con euristica solo ammissibile (non consistente): potresti trovare uno stato già espanso con un percorso più economico e doverlo riaprire.

4 Ricerca Avversariale (Giochi)

4.1 Minimax

Giocare contro un avversario razionale

Nei giochi a due giocatori (scacchi, tris, dama), ci sono due agenti con obiettivi opposti: **MAX** vuole massimizzare il valore finale, **MIN** vuole minimizzarlo. L'idea di Minimax è semplice: MAX deve scegliere la mossa *assumendo che MIN giochi in modo perfetto*.

Si costruisce l'albero di gioco fino agli stati terminali (o a una profondità limite), si assegnano i valori di utilità agli stati terminali, poi si "propagano" i valori verso il basso:

- Ai nodi MAX: prendo il **massimo** tra i figli.
- Ai nodi MIN: prendo il **minimo** tra i figli.

Esempio: Minimax su un albero piccolo

Albero con radice MAX, due figli MIN, quattro nipoti terminali con valori $[3, 5]$ (sotto MIN di sinistra) e $[2, 9]$ (sotto MIN di destra).

MIN di sinistra sceglie $\min(3, 5) = 3$.

MIN di destra sceglie $\min(2, 9) = 2$.

MAX sceglie $\max(3, 2) = 3$.

Quindi MAX va a sinistra, sapendo che MIN lo porterà al valore 3. Andare a destra darebbe 2 (peggio per MAX).

Nota: il valore 9 non viene mai raggiunto – MIN non lo permetterebbe.

Analogia: Giocare a scacchi come un maestro

Un giocatore esperto non pensa solo alla propria prossima mossa, ma ragiona: “se faccio questa mossa, l’avversario risponderà con la sua mossa migliore, e poi io risponderò con la mia, e così via”. Minimax formalizza esattamente questo ragionamento ricorsivo.

4.2 Potatura Alpha-Beta

Tagliare i rami inutili

Minimax è corretto ma lento: per gli scacchi, $b \approx 35$ mosse per turno e $m \approx 80$ turni $\Rightarrow 35^{80}$ nodi – impossibile. La potatura **Alpha-Beta** mantiene due variabili:

- α : il **miglior valore** che MAX può garantirsi nel percorso corrente (inizia a $-\infty$).
- β : il **peggior valore** che MIN può garantirsi nel percorso corrente (inizia a $+\infty$).

Regola: se in un nodo MIN il valore scende sotto α , MAX non sceglierà mai quel ramo $\Rightarrow$ **pota**. Se in un nodo MAX il valore sale sopra β , MIN non permetterà mai quel ramo $\Rightarrow$ **pota**.

Il risultato è **identico a Minimax**: la potatura non cambia la decisione, solo riduce il lavoro.

Esempio: Alpha-Beta passo per passo

Stessa struttura di prima: radice MAX, due MIN, foglie [3, 5, 2, 9].

Stato iniziale: $\alpha = -\infty$, $\beta = +\infty$.

1. Esploro il MIN di sinistra. Prima foglia = 3. $\beta \leftarrow 3$ (il MIN di sinistra può garantire al più 3 a MAX).
2. Seconda foglia = 5. MIN sceglie $\min(3, 5) = 3$. Valore del nodo MIN = 3.
3. Risalgo al MAX: $\alpha \leftarrow 3$ (ora MAX sa di poter avere almeno 3).
4. Esploro il MIN di destra. Prima foglia = 2. $\beta_{\text{locale}} \leftarrow 2$. Ma $2 < \alpha = 3$!
 $\Rightarrow$ **POTA** la seconda foglia (9) del MIN di destra: MAX non sceglierà mai questo ramo (il MIN di destra può dargli al più 2, mentre a sinistra ha già 3).
5. Risultato: MAX sceglie sinistra con valore 3. La foglia 9 non è mai stata esplorata.

Da ricordare: Complessità Alpha-Beta

- Caso **migliore** (ordine perfetto delle mosse): $O(b^{m/2})$ – **raddoppia** la profondità esplorabile a parità di tempo!
- Caso **medio** (ordine casuale): $O(b^{3m/4})$
- Caso **peggiore** (ordine inverso): $O(b^m)$ – come Minimax

Per questo l’ordinamento delle mosse è fondamentale: provare prima le mosse più promettenti massimizza la potatura.

4.3 Tecniche avanzate

Quando non puoi esplorare tutto l'albero

Negli scacchi non puoi arrivare agli stati terminali. Si usa una **funzione di valutazione EVAL**:

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

Combinazione lineare di feature (vantaggio di materiale, controllo del centro, sicurezza del re...). EVAL è una *stima* applicata agli stati non terminali; la Utility è il valore *esatto* degli stati terminali.

Effetto orizzonte: l'algoritmo non vede le minacce oltre il limite di profondità. Soluzione: **ricerca di quiescenza** – continua oltre il limite nelle posizioni “turbolente” (prese, scacchi) finché la posizione si stabilizza.

Tabelle di trasposizione: se la stessa posizione compare da percorsi diversi (es. mosse in ordine diverso), non ricalcolarla. Usa una tabella hash: recupero in $O(1)$.

5 Problemi di Soddisfacimento di Vincoli (CSP)

5.1 Cos'è un CSP

Struttura di un CSP

Un CSP è una tripla $\langle X, D, C \rangle$: variabili X , domini D (valori possibili per ciascuna variabile), vincoli C (relazioni che i valori devono rispettare). Una **soluzione** è un assegnamento completo (tutte le variabili hanno un valore) e consistente (nessun vincolo violato).

Differenza dalla ricerca classica: in un CSP lo spazio degli stati è **strutturato**. I vincoli ci permettono di **propagare** l'informazione: se assegno un valore a una variabile, posso immediatamente eliminare valori incompatibili dalle variabili adiacenti. Questo porta enormi porzioni dello spazio di ricerca prima ancora di esplorarle.

Esempio: Colorazione della mappa australiana

Variabili: WA (Western Australia), NT (Northern Territory), SA (South Australia), Q (Queensland), NSW (New South Wales), V (Victoria), T (Tasmania).

Dominio: {rosso, verde, blu}.

Vincoli: regioni adiacenti devono avere colori diversi ($WA \neq NT$, $WA \neq SA$, $NT \neq SA$, $NT \neq Q$, $SA \neq Q$, $SA \neq NSW$, $SA \neq V$, $Q \neq NSW$, $NSW \neq V$).

Il grafo dei vincoli ha un arco tra ogni coppia di regioni adiacenti. SA è connessa a 5 regioni – ha il dominio che si riduce più velocemente. Soluzione: WA=rosso, NT=verde, SA=blu, Q=rosso, NSW=verde, V=rosso, T=qualsiasi.

Esempio: Sudoku come CSP

81 variabili ($S_{i,j}$ per ogni cella). Dominio = $\{1, \dots, 9\}$. Vincoli: tutte le celle di ogni riga diversa (Alldiff), idem per colonne e blocchi 3×3 . Molte istanze si risolvono *solo per propagazione dei vincoli*, senza backtracking!

5.2 Propagazione dei vincoli

AC-3: arco-consistenza

Una variabile X_i è **arco-consistente** rispetto a X_j se per ogni valore v nel dominio di X_i esiste almeno un valore compatibile nel dominio di X_j . Se v non ha supporto, lo eliminiamo dal dominio di X_i .

L'algoritmo **AC-3** mantiene una coda di archi (X_i, X_j) da verificare. Quando eliminiamo un valore da D_i , reinserisci tutti gli archi (X_k, X_i) nella coda – perché potrebbero aver perso il loro supporto.

Complessità: $O(n^2d^3)$ dove n = variabili, d = dimensione massima del dominio.

Esempio: AC-3 sul Sudoku

Considera una riga del Sudoku con la cella $X_1 = 5$ (assegnata). AC-3 rimuove 5 dai domini di tutte le altre celle della stessa riga, colonna e blocco. Se dopo questa rimozione un'altra cella ha dominio con un solo valore, anche quello viene propagato, e così via a cascata. Spesso questo è sufficiente a completare il puzzle!

5.3 Backtracking con euristiche

Come scegliere cosa assegnare

Il backtracking puro prova ogni valore per ogni variabile in ordine arbitrario – lentissimo. Tre euristiche rendono il backtracking molto più efficiente:

1. **MRV (Minimum Remaining Values)**: scegli la variabile con il **dominio più piccolo**. Principio *fail-first*: è meglio scoprire presto che una variabile non ha soluzione (e fare backtrack) piuttosto che sprecare tempo assegnando altre variabili. Intuizione: la variabile con dominio minore è quella “più vincolata”, quindi quella su cui il backtracking è più probabile – meglio trattarla subito.
2. **Euristica di Grado**: tra variabili con MRV uguale, scegli quella con **più vincoli sulle variabili non ancora assegnate**. Riduce il fattore di ramificazione futuro.
3. **Least Constraining Value**: dopo aver scelto la variabile, assegna il valore che **elimina meno valori dai domini delle variabili vicine**. Lascia più opzioni aperte per il futuro (principio contrario a MRV: per i valori, vogliamo essere ottimisti).

Forward Checking e MAC

Forward Checking: ogni volta che assegni $X_i = v$, rimuovi i valori incompatibili dai domini dei vicini non assegnati. Se un dominio si svuota, backtrack subito (senza espandere ulteriormente).

MAC (Maintaining Arc Consistency): più potente; dopo ogni assegnamento, applica AC-3 all'intero CSP. Pota molto di più ma costa di più per ogni passo.

Conflict-Directed Backjumping: invece di tornare sempre alla variabile precedente (backtracking cronologico), costruisci il **conflict set** (le variabili già assegnate che causano il fallimento) e salta direttamente all'ultima di esse. Molto più efficiente su problemi complessi.

6 Logica Proposizionale

6.1 Perché la logica

Ragionare con la conoscenza

Un agente “basato sulla conoscenza” (KB agent) non reagisce a stimoli con regole pre-programmate: mantiene una **base di conoscenza (KB)** – un insieme di formule logiche – e usa la logica per dedurre nuove informazioni e scegliere le azioni. Il ciclo è: percepisce (TELL), ragiona (ASK), agisce.

La logica proposizionale è il livello base: afferma fatti semplici (“P è vero”) e li collega con connettivi ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$).

Esempio: Il Mondo del Wumpus

Una griglia 4×4 . Ci sono:

- **Wumpus**: puzza nelle celle adiacenti (Fetore). Se ci vai, muori.
- **Pozzi**: brezza nelle celle adiacenti. Se ci cadi, muori.
- **Oro**: scintilla nella cella in cui si trova. Prendilo e scappa.

L’agente parte da (1,1). Sente brezza in (1,2) $\Rightarrow$ c’è un pozzo in (1,3) o (2,2). Sente fetore in (2,1) $\Rightarrow$ Wumpus in (1,1) o (3,1) o (2,2). Usando la KB: siccome l’agente è partito da (1,1) e non è morto, (1,1) non è un pozzo né il Wumpus è lì. Quindi per esclusione...

La logica proposizionale ci permette di ragionare su questi fatti in modo formale e rigoroso.

Concetti fondamentali

- **Modello**: un’assegnazione di valori di verità a tutte le proposizioni. Con n proposizioni ci sono 2^n modelli possibili.
- **Conseguenza logica**: $KB \models \alpha$ se α è vera in *tutti* i modelli in cui KB è vera.
- **Tautologia**: vera in tutti i modelli (es: $A \vee \neg A$).
- **Contraddizione**: falsa in tutti i modelli (es: $A \wedge \neg A$).
- **Soddisfacibile**: vera in almeno un modello.

6.2 Dimostrazione di teoremi

CNF e Risoluzione per refutazione

Per dimostrare che $KB \models \alpha$, usiamo la **dimostrazione per assurdo**: supponiamo $\neg\alpha$ e mostriamo che porta a contraddizione.

Prima convertiamo tutto in **CNF** (Forma Normale Congiuntiva): congiunzione di clausole, dove ogni clausola è una disgiunzione di letterali. Ogni formula si può convertire in CNF.

Poi applichiamo la **regola di risoluzione**: da $(A \vee C)$ e $(\neg C \vee B)$ si deriva $(A \vee B)$. I letterali complementari C e $\neg C$ si annullano.

Aggiungiamo $\neg\alpha$ alla KB in CNF e applichiamo ripetutamente la risoluzione. Se deriviamo la **clausola vuota** $\square$ (contraddizione) $\Rightarrow KB \models \alpha$.

Esempio: Risoluzione: esempio completo

$KB = \{(A \vee B), (\neg A \vee C), \neg B\}$. Vogliamo dimostrare C .

Aggiungiamo $\neg C$: $KB' = \{(A \vee B), (\neg A \vee C), \neg B, \neg C\}$.

1. Da $(A \vee B)$ e $\neg B$: risolviamo su $B \Rightarrow A$.

2. Da $(\neg A \vee C)$ e $\neg C$: risolviamo su $C \Rightarrow \neg A$.

3. Da A e $\neg A$: **clausola vuota** $\square$. Contraddizione!

$\Rightarrow KB \models C$. ✓

6.3 Clausole di Horn e Forward/Backward Chaining

Perché le clausole di Horn sono speciali

Una clausola di Horn ha **al più un letterale positivo**: $P_1 \wedge P_2 \wedge \dots \wedge P_k \Rightarrow Q$ (clausola **definita**) oppure $P_1 \wedge \dots \wedge P_k \Rightarrow \text{false}$ (clausola **obiettivo**).

Le clausole di Horn sono potenti perché l'inferenza su di esse è **decidibile in tempo lineare**: si usa Forward Chaining o Backward Chaining invece della risoluzione generale.

Forward Chaining (guidato dai dati): parte dai fatti noti, applica le regole le cui premesse sono soddisfatte, aggiunge le conclusioni, ripete fino al punto fisso. Come l'investigatore che raccoglie indizi e trova nuove connessioni.

Backward Chaining (guidato dall'obiettivo): parte dal goal, cerca le regole che lo provano, trasforma le premesse in sotto-obiettivi, ricorsivamente. Come l'avvocato difensore che parte dalla tesi e cerca le prove per supportarla. È la base di **Prolog**.

Esempio: Forward Chaining

KB: "A è un uccello" ($\text{Uccello}(A)$); "Gli uccelli volano" ($\text{Uccello}(x) \Rightarrow \text{Vola}(x)$); "Chi vola è leggero" ($\text{Vola}(x) \Rightarrow \text{Leggero}(x)$).

Query: A è leggero?

Iterazione 1: so $\text{Uccello}(A)$ + regola 1 $\Rightarrow$ derivo $\text{Vola}(A)$.

Iterazione 2: so $\text{Vola}(A)$ + regola 2 $\Rightarrow$ derivo $\text{Leggero}(A)$.

Goal raggiunto!

7 Logica del Primo Ordine (FOL)

7.1 Perché la FOL

I limiti della logica proposizionale

La logica proposizionale è limitata: devo scrivere una proposizione separata per ogni fatto. "Mario è studente", "Luigi è studente", "Maria è studente"... Non posso dire "tutti gli studenti passano gli esami". La FOL risolve questo con **variabili e quantificatori**.

Nella FOL esistono: **costanti** (oggetti specifici: Mario, Roma), **variabili** (x, y, z), **predicati** (relazioni: $\text{Studia}(x)$, $\text{Fratello}(x, y)$), **funzioni** (mappano oggetti: $\text{Padre}(x)$), **quantificatori** ($\forall, \exists$).

Esempio: FOL in pratica: relazioni di parentela

- “Riccardo ha un fratello”: $\exists x \text{ Fratello}(x, \text{Riccardo})$
- “Tutti gli umani sono mortali”: $\forall x \text{ Umano}(x) \Rightarrow \text{Mortale}(x)$
- “Socrate è umano”: $\text{Umano}(\text{Socrate})$
- Da queste due: $\text{Mortale}(\text{Socrate})$ (via Modus Ponens)
- “Qualcuno ama tutti”: $\exists x \forall y \text{ Ama}(x, y)$
vs “ognuno ama qualcuno”: $\forall x \exists y \text{ Ama}(x, y)$
– L’ordine dei quantificatori cambia il significato!

Da ricordare: Regola pratica per i quantificatori

- $\forall x$ si abbina con $\Rightarrow$: “per ogni x , se x è umano allora...”
- $\exists x$ si abbina con $\wedge$: “esiste un x tale che x è umano e x vola”
- Sbagliato: $\exists x \text{ Umano}(x) \Rightarrow \text{Mortale}(x)$ – è vero anche se nessun umano esiste!

7.2 Unificazione

Trovare la sostituzione giusta

L’unificazione è l’operazione che trova la sostituzione θ che rende due espressioni FOL identiche. Serve per applicare le regole a fatti concreti.

Il **MGU** (Most General Unifier) è la sostituzione più “generale” (con meno vincoli) che unifica due espressioni.

Esempio: Unificazione passo per passo

Regola: $\text{Conosce}(x, y) \wedge \text{Conosce}(y, z) \Rightarrow \text{Conosce}(x, z)$

Fatto1: $\text{Conosce}(\text{Mario}, \text{Luigi})$

Fatto2: $\text{Conosce}(\text{Luigi}, \text{Maria})$

Unificazione di Fatto1 con $\text{Conosce}(x, y)$: $\theta_1 = \{x/\text{Mario}, y/\text{Luigi}\}$

Unificazione di Fatto2 con $\text{Conosce}(y, z)$ usando θ_1 ($y=\text{Luigi}$): $\theta_2 = \{z/\text{Maria}\}$

Deriviamo: $\text{Conosce}(\text{Mario}, \text{Maria})$. Mario conosce Maria transitivamente!

Esempio: Quando l’unificazione fallisce

$\text{UNIFY}(\text{Conosce}(\text{Giovanni}, x), \text{Conosce}(x, \text{Elisabetta}))$

Serve $x = \text{Giovanni}$ (dal primo argomento) e $x = \text{Elisabetta}$ (dal secondo): impossibile, x non può valere due cose diverse.

Soluzione: **standardizzare le variabili** rinominando x in una delle due espressioni:

$\text{UNIFY}(\text{Conosce}(\text{Giovanni}, x), \text{Conosce}(z, \text{Elisabetta})) = \{x/\text{Elisabetta}, z/\text{Giovanni}\}$.

Successo!

7.3 Modus Ponens Generalizzato e Skolemizzazione

MPG e conversione in CNF

Il **Modus Ponens Generalizzato (MPG)** estende il Modus Ponens alle formule con variabili: se conosco $P_1, \dots, P_k$ e ho la regola $P'_1 \wedge \dots \wedge P'_k \Rightarrow Q$ con unificatore θ tale che $P_i\theta = P'_i\theta$, allora derivo $Q\theta$.

Per la risoluzione in FOL, devo convertire in CNF. Il passo cruciale è la **Skolemizzazione**: elimino i quantificatori esistenziali sostituendo la variabile con una costante/funzione di Skolem.

Esempio: $\exists x \text{ Madre}(x, \text{Mario})$ diventa $\text{Madre}(c_1, \text{Mario})$ (c_1 è una nuova costante di Skolem). Se c'è un $\forall$ esterno: $\forall x \exists y \text{ Ama}(x, y)$ diventa $\forall x \text{ Ama}(x, f(x))$ dove f è una *funzione* di Skolem (il “qualcuno amato” dipende da x).

8 Ontologie e Rappresentazione della Conoscenza

Cosa sono le ontologie

Un'ontologia è una **specifica formale di una concettualizzazione**: definisce i concetti di un dominio e le relazioni tra loro. Le relazioni fondamentali sono:

isa (is-a): relazione di sottoclasse. Cane **isa** Animale significa che i cani sono un tipo di animali. Permette l'ereditarietà delle proprietà.

member-of: relazione di istanza. Fido **member – of** Cane significa che Fido è un individuo della classe Cani.

part-of: relazione di composizione. Ruota **part – of** Auto.

Esempio: Tassonomia degli animali

- Animale $\supseteq$ Mammifero $\supseteq$ Cane; Fido $\in$ Cane.
- **Partizione**: Mammifero, Rettile, Uccello, Pesce sono *disgiunti* (nessun animale è sia mammifero che rettile) ed *esaustivi* rispetto agli Animali (ogni animale è in almeno una categoria).
- Proprietà di classe: “I mammiferi allattano i cuccioli” – vale per tutta la classe, non solo per qualche istanza.

Situation Calculus e il Frame Problem

Il **Situation Calculus** è un linguaggio logico per ragionare su azioni e i loro effetti nel tempo. I concetti chiave:

- **Situazione**: storia delle azioni eseguite fino a un certo momento.
- **Fluente**: proprietà che può cambiare valore (es: la posizione di un blocco).
- **Do(azione, situazione)**: il risultato di eseguire un'azione in una situazione.

Il **Frame Problem**: come sapere cosa *non* cambia dopo un'azione? Se sposto un blocco da A a B, il colore del blocco rimane lo stesso – ma devo specificarlo esplicitamente per ogni fluente! Gli **assiomi dello stato successore** risolvono questo: un fluente è vero dopo un'azione *se* l'azione lo rende vero, *oppure* era già vero e l'azione non lo ha reso falso.

Anomalia di Sussman: nei piani lineari (perseguire un sotto-obiettivo alla volta), il perseguimento di un obiettivo può disfare il lavoro fatto per un altro. Soluzione: *interleaving* dei passi (pianificazione non lineare).

9 Machine Learning

9.1 Alberi di Decisione

Come costruire un albero di decisione

L'algoritmo di Hunt (base di ID3 e C4.5) costruisce l'albero ricorsivamente:

1. Se tutti gli esempi nel nodo corrente appartengono alla stessa classe $\Rightarrow$ crea una foglia con quella classe.
2. Altrimenti: scegli l'attributo che "divide meglio" gli esempi (massimizza il guadagno di informazione).
3. Crea un figlio per ogni valore dell'attributo, assegna gli esempi ai figli, ripeti.

Esempio: Giocare a tennis: albero di decisione

Dataset: 14 giornate. Attributi: *Meteo* (Sole/Nuvoloso/Pioggia), *Temperatura* (Alta/Media/Bassa), *Umidità* (Alta/Normale), *Vento* (Forte/Debole). Classe: *Gioca* (Sì/No).

L'entropia della radice: $H = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx 0.940$.

Per l'attributo *Meteo*:

- Sole (5 esempi: 2 sì, 3 no): $H = 0.971$
- Nuvoloso (4 esempi: 4 sì, 0 no): $H = 0$ (puro!)
- Pioggia (5 esempi: 3 sì, 2 no): $H = 0.971$

$\text{Gain}(\text{Meteo}) = 0.940 - (\frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971) \approx 0.247$

Il ramo *Nuvoloso* diventa subito una foglia "Gioca=Sì". Gli altri rami si espandono ulteriormente con altri attributi.

Overfitting e pruning

Un albero costruito completamente su dati di training si adatta al rumore: memorizza le eccezioni invece di catturare i pattern generali (**overfitting**). Due soluzioni:

Pre-Pruning: interrompi la costruzione se il guadagno di informazione è sotto una soglia. Rischio: ci si ferma troppo presto (**underfitting**).

Post-Pruning: costruisci l'albero completo, poi *pota* i rami sostituendoli con foglie della classe più frequente. Usa un **validation set** separato dal training: pota se le prestazioni sul validation set migliorano. È più robusto del pre-pruning.

Il **Rasoio di Occam**: a parità di errore, il modello più semplice è preferibile. Un albero piccolo generalizza meglio di uno grande.

Metriche di valutazione

La **matrice di confusione** riassume le previsioni del modello:

		Previsto	
		Positivo	Negativo
Reale	Positivo	TP	FN
	Negativo	FP	TN

Accuratezza = $\frac{TP+TN}{TP+TN+FP+FN}$. **Attenzione:** con classi sbilanciate, l'accuratezza è ingannevole! Se il 99% dei campioni è negativo, un classificatore che dice sempre "negativo" ha 99% di accuratezza ma è inutile.

10 Reti Neurali

10.1 Il Percettrone

Come funziona un singolo neurone

Il percettrone prende n input $x_1, \dots, x_n$ con pesi $w_1, \dots, w_n$ e un bias b . Calcola la somma pesata $\text{net} = \sum_i w_i x_i + b$ e applica la funzione di attivazione $f(\text{net})$.

Geometricamente: i pesi definiscono un **iperpiano** nello spazio degli input. Gli esempi su un lato dell'iperpiano sono classe 1, gli altri classe 0. Per questo il percettrone **non può risolvere XOR**: XOR non è linearmente separabile (non esiste un piano che separa le due classi).

Esempio: Calcolo forward pass con numeri

Input: $x_1 = 1, x_2 = 0$. Pesi: $w_1 = 0.5, w_2 = -0.3$. Bias: $b = 0.1$.

$\text{net} = 0.5 \cdot 1 + (-0.3) \cdot 0 + 0.1 = 0.6$

Con funzione sigmoide: $\sigma(0.6) = \frac{1}{1+e^{-0.6}} \approx \frac{1}{1+0.549} \approx 0.646$

L'output è ≈ 0.646 . Se soglia = 0.5 $\Rightarrow$ classe 1.

Esempio: Regola Delta: aggiornamento dei pesi

Supponi: output desiderato $d = 1$, output ottenuto $o = 0.646$, learning rate $\eta = 0.1$.

Errore: $d - o = 1 - 0.646 = 0.354$

Aggiornamento:

$$w_1 \leftarrow 0.5 + 0.1 \cdot 0.354 \cdot 1 = 0.535$$

$$w_2 \leftarrow -0.3 + 0.1 \cdot 0.354 \cdot 0 = -0.3 \quad (\text{input zero, nessun aggiornamento})$$

$$b \leftarrow 0.1 + 0.1 \cdot 0.354 \cdot 1 = 0.135$$

Alla prossima iterazione, l'output sarà più vicino a 1.

10.2 MLP e Backpropagation

Perché servono più strati

Un singolo perceptrone può classificare solo funzioni linearmente separabili. Un **MLP** (Multi-Layer Perceptron) con almeno un hidden layer può approssimare qualsiasi funzione continua (**Universal Approximation Theorem**). Gli hidden layer imparano *rappresentazioni intermedie* – caratteristiche sempre più astratte degli input.

L'architettura:

- **Input layer:** un neurone per feature (identità).
- **Hidden layer(s):** neuroni con sigmoide. Trasformano l'input in rappresentazioni utili.
- **Output layer:** produce la classificazione finale.
- Per k classi: k neuroni di output (uno per classe).

Backpropagation: come impara la rete

Il training è in due fasi, ripetute per ogni esempio o mini-batch:

1. **Forward Pass:** calcola l'output di ogni neurone layer per layer, fino all'output. Calcola l'errore $E = \frac{1}{2} \sum_i (t_i - y_i)^2$.
2. **Backward Pass:** propaga l'errore *all'indietro* usando la **regola della catena**. Calcola il gradiente $\frac{\partial E}{\partial w}$ per ogni peso e aggiornalo nella direzione che riduce l'errore:

$$w \leftarrow w - \eta \frac{\partial E}{\partial w}$$

La *regola della catena* permette di calcolare come ogni peso influenza l'errore finale, anche se il peso è molti layer “lontano” dall'output.

Attenzione: Problemi pratici dell'addestramento

- **Vanishing gradient:** con sigmoidi profonde, il gradiente si azzerava verso i layer iniziali (la derivata della sigmoide è al massimo 0.25). Soluzione: **ReLU** ($f(x) = \max(0, x)$, derivata = 1 per $x > 0$).
- **Learning rate:** troppo grande $\Rightarrow$ divergenza (i pesi oscillano). Troppo piccolo $\Rightarrow$ convergenza lentissima. Soluzione: Adam, RMSProp (adattativi).
- **Overfitting:** la rete memorizza il training set. Soluzione: dropout (disattiva casualmente alcuni neuroni durante il training), regolarizzazione L1/L2.

11 Apprendimento per Rinforzo

11.1 Il framework MDP

Imparare dall'interazione

Nel reinforcement learning, un agente interagisce con un ambiente: sceglie azioni, riceve reward, aggiorna la sua strategia. Non c'è un dataset di esempi etichettati: l'agente scopre cosa funziona attraverso l'esperienza.

Il framework formale è il **Markov Decision Process (MDP)**: $\langle S, A, T, R, \gamma \rangle$.

- S : stati. A : azioni. $T(s, a, s')$: probabilità di transizione.
- $R(s, a, s')$: reward ricevuto. γ : fattore di sconto ($0 \leq \gamma < 1$).
- **Proprietà di Markov**: il futuro dipende solo dallo stato corrente, non dalla storia.

Analogia: Il topo nel labirinto

Un topo in un labirinto esplora a caso inizialmente. Trova il formaggio (reward +10) in alcune celle, sbatte contro i muri (-1 per ogni mossa), cadono trappole (-10). Col tempo impara il percorso migliore verso il formaggio, evitando le trappole. Questa è l'essenza del RL.

Policy, Value Function e Q-value

Policy π : la strategia dell'agente – data uno stato, quale azione compiere.

Value function $V^\pi(s)$: valore atteso dei reward futuri (scontati) partendo da s e seguendo π .

Q-value $Q^\pi(s, a)$: valore atteso partendo da s , facendo a , poi seguendo π .

Il γ (fattore di sconto) gestisce l'orizzonte temporale: $\gamma = 0$ considera solo il reward immediato (miope), $\gamma \rightarrow 1$ considera tutti i reward futuri ugualmente (lungimirante). In pratica $\gamma \approx 0.9-0.99$.

La **policy ottimale**: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

11.2 Algoritmi di apprendimento

Value Iteration

Se conosco T e R (model-based), posso calcolare V^* iterativamente:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V_k(s')]$$

Inizia con $V_0 = 0$ per tutti gli stati. A ogni iterazione, il valore “si propaga” dagli stati con reward alto agli stati vicini. Converge a V^* perché la funzione è una contrazione (con $\gamma < 1$).

Esempio: Value Iteration su una griglia 3x3

Griglia 3x3. Goal in (3,3) con reward +1, trappola in (2,2) con reward -1. $\gamma = 0.9$.

Iterazione 0: tutti i valori = 0.

Iterazione 1: $V(3,3) \leftarrow 1$, $V(2,2) \leftarrow -1$ (reward immediato).

Iterazione 2: le celle adiacenti a (3,3) ricevono $\gamma \cdot 1 = 0.9$; le adiacenti a (2,2) ricevono $\gamma \cdot (-1) = -0.9$.

Iterazione 3: le celle a distanza 2 da (3,3) ricevono $\gamma^2 \approx 0.81 \dots$

Dopo alcune iterazioni emerge la policy ottimale: ogni cella “punta” verso (3,3) evitando (2,2).

Q-Learning: imparare senza modello

Q-Learning è **model-free**: non conosce T e R . Impara direttamente dall'esperienza. Ad

ogni step (s, a, r, s') :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Dove α è il learning rate. La formula dice: aggiorna $Q(s, a)$ nella direzione del **target** $r + \gamma \max_{a'} Q(s', a')$ (reward immediato + valore futuro stimato).

Q-Learning converge a Q^* con sufficiente esplorazione (tutte le coppie stato-azione visitate infinite volte). È **off-policy**: converge a Q^* indipendentemente dalla policy di esplorazione.

Esplorazione ε -greedy: con probabilità ε scegli un'azione casuale (esplorazione); con probabilità $1 - \varepsilon$ scegli $\arg \max_a Q(s, a)$ (sfruttamento). ε decresce nel tempo.

12 Riepilogo – Mappa concettuale

Come collegare tutto

Il corso segue un filo logico: come costruire agenti intelligenti che risolvono problemi sempre più complessi.

Ricerca (Cap. 2–4): l'agente conosce lo spazio degli stati e cerca un cammino. Blind search quando non ha informazioni; informed search (A^*) quando ha un'euristica; adversarial search nei giochi.

CSP (Cap. 5): l'agente deve trovare un assegnamento consistente. La struttura del problema (vincoli) permette di potare enormi porzioni dello spazio prima di esplorarle.

Logica (Cap. 6–8): l'agente ragiona con una base di conoscenza formale. Proposizionale per il mondo semplice, FOL per generalizzare su oggetti e relazioni.

Ontologie (Cap. 9): come organizzare la conoscenza del dominio in modo riusabile e strutturato.

ML, Reti Neurali, RL (Cap. 10–12): l'agente *apprende* dalla esperienza invece di essere programmato esplicitamente. Supervisioned learning (alberi, reti), reinforcement learning (MDP, Q-Learning).

Da ricordare: Formule chiave da portare all'esame

Concetto	Formula / Proprietà chiave
A*	$f(n) = g(n) + h(n)$; ottimale se h ammissibile
Ammissibilità	$h(n) \leq h^*(n)$ (non sovrastima mai)
Manhattan	$\sum_i x_i - x_i^* + y_i - y_i^* $
IDDFS	Spazio $O(bd)$, Tempo $O(b^d)$ – il migliore blind
Alpha-Beta	Caso migliore $O(b^{m/2})$ – ordine perfetto delle mosse
AC-3	Complessità $O(n^2d^3)$
Entropia	$H(t) = -\sum_i p(i t) \log_2 p(i t)$
Information Gain	$\text{Gain} = H(\text{parent}) - \sum_v \frac{ D_v }{ D } H(D_v)$
Regola Delta	$\Delta w_j = \eta \cdot (d - o) \cdot x_j$
Backprop	$\Delta w = -\eta \frac{\partial E}{\partial w}$ (discesa del gradiente)
Bellman	$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R + \gamma V^*(s')]$
Q-Learning	$Q(s, a) \leftarrow \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

Da ricordare: Trappole frequenti all'esame

- BFS ottimale **solo** con costi uniformi. Con costi diversi $\Rightarrow$ UCS.
- Una clausola di Horn può avere **zero** letterali positivi (clausola obiettivo) o **uno** (clausola definita). NON può averne due o più.
- MPG (Modus Ponens Generalizzato) richiede che le premesse della KB *unifichino* con le premesse della regola – non devono essere identiche.
- Alpha-Beta dà lo **stesso risultato** di Minimax: la potatura non cambia la decisione, solo il tempo di calcolo.
- $\forall x P(x) \Rightarrow Q(x)$ si legge “per ogni x , se $P(x)$ allora $Q(x)$ ” – $\Rightarrow$ si usa con $\forall$, **non** $\wedge$.
- Nella Skolemizzazione, se $\exists$ è all'interno di un $\forall y$, usa una **funzione** di Skolem $f(y)$, non una semplice costante.
- Q-Learning converge a Q^* anche se non segui la policy ottimale durante l'esplorazione (off-policy). Value Iteration invece richiede il modello completo.